Different Types of Prompting

1. Importing required libraries

pip install langchain langchain-openai langchain-groq --quiet

from langchain_core.prompts import ChatPromptTemplatefrom langchain_groq import ChatGroq

from google.colab import userdatagroq_api_key = userdata.get('GROQ_API_KEY')llm = ChatGroq(model="openai/gpt-oss-120b",   temperature=0.2,   api_key=groq_api_key)

2. Zero shot prompting

```
zero_shot_prompt = ChatPromptTemplate.from_messages([
    ("human", "Explain what a Python dictionary is and give an example.")
])

chain = zero_shot_prompt | llm
print(chain.invoke({}).content)
```

Output

What a Python dictionary is

A *dictionary* (often called a **dict**) is one of Python's built-in data structures. It stores a collection of **key-value pairs** where:

Feature	Description
Key	Must be *hashable* (e.g., strings, numbers, tuples of immutable objects). Keys are unique – adding a duplicate key overwrites the previous value.
Value	Can be any Python object (including other dictionaries, lists, functions, etc.).
Mutable	You can add, remove, or change entries after the dictionary is created.
Lookup speed	Average-case O(1) time for retrieving a value by its key, because a dict is implemented as a hash table.
Order	Since Python 3.7 the insertion order is preserved (iteration yields items in the order they were added).

In everyday language a dictionary is like a real-world lookup table: you give it a *key* (the word you're looking up) and it returns the *value* (the definition).

Basic syntax

```python
# Literal syntax using curly braces
my_dict = {
    "apple": 3,          # key: "apple", value: 3
    "banana": 5,
    "orange": 2
}
```

You can also create an empty dict and fill it later:

```python
my_dict = {}          # empty dictionary
my_dict["pear"] = 4   # add a new key-value pair
```

Common operations

```python
# Access a value (raises KeyError if the key is missing)
num_apples = my_dict["apple"]          # → 3

# Safer access with a default
num_mangoes = my_dict.get("mango", 0)  # → 0 (default because "mango" not present)

# Add / update a value
my_dict["banana"] = 6   # updates existing key

# Remove a key-value pair
removed = my_dict.pop("orange")  # returns the removed value (2)

# Check for existence of a key
if "pear" in my_dict:
    print("We have pears!")
```

```
# Iterate over keys, values, or items
for fruit in my_dict:              # iterates over keys
    print(fruit, my_dict[fruit])

for fruit, count in my_dict.items():  # iterates over (key, value) tuples
    print(fruit, count)
```

A concrete example

Suppose you're writing a small program to keep track of a grocery store's inventory. A dictionary can map each product name to the quantity in stock:

```python
# inventory.py
def main():
    # Initial inventory
    inventory = {
        "bread": 12,
        "milk": 8,
        "eggs": 30,
        "cheese": 5,
    }

    # Show current stock
    print("Current inventory:")
    for item, qty in inventory.items():
        print(f"  {item:6} → {qty}")

    # Customer buys 2 loaves of bread
    inventory["bread"] -= 2

    # New shipment arrives: 10 more milk cartons
    inventory["milk"] = inventory.get("milk", 0) + 10

    # Add a brand-new product
    inventory["yogurt"] = 15

    print("\nAfter updates:")
    for item, qty in inventory.items():
        print(f"  {item:6} → {qty}")
```

```
if __name__ == "__main__":
    main()
```

Output

```
Current inventory:
  bread  → 12
  milk   → 8
  eggs   → 30
  cheese → 5

After updates:
  bread  → 10
  milk   → 18
  eggs   → 30
  cheese → 5
  yogurt → 15
```

In this example:

* The **keys** are the product names (`"bread"`, `"milk"`, …).
* The **values** are the integer quantities in stock.
* We easily read, modify, add, and remove entries using the dictionary's built-in operations.

Quick recap

* A Python dictionary is a mutable, hash-based collection of unique keys mapped to values.
* It provides fast O(1) average lookup, insertion, and deletion.
* Syntax: `{key1: value1, key2: value2, ...}` or `dict()` for an empty dict.
* Use methods like `.get()`, `.pop()`, `.items()`, `.keys()`, and `.values()` to work with the data.

That's the essence of a Python dictionary—an extremely handy structure for any situation where you need to associate one piece of data (the key) with another (the value).

3. One shot prompting

```
one_shot_prompt = ChatPromptTemplate.from_messages([
    ("human", """
Example:
Q: What is a list in Python?
A: A list is an ordered, mutable collection of elements.

Now answer:
Q: What is a tuple in Python?
""")
])

chain = one_shot_prompt | llm
print(chain.invoke({}).content)
```

Output

A tuple in Python is an ordered, **immutable** collection of elements. It can hold items of any type, is defined using parentheses (e.g., `my_tuple = (1, "a", 3.14)`) or without them (`my_tuple = 1, "a", 3.14`), and its contents cannot be changed after creation.

4. Few shot prompting

```
few_shot_prompt = ChatPromptTemplate.from_messages([
    ("human", """
Q: What is a stack?
A: A linear data structure that follows LIFO.

Q: What is a queue?
A: A linear data structure that follows FIFO.

Q: What is a linked list?
""")
])

chain = few_shot_prompt | llm
print(chain.invoke({}).content)
```

Output

Q: What is a linked list?
A: A linked list is a linear data structure composed of a sequence of *nodes*, where each node stores a data element and one or more references (pointers) to other nodes. In a singly linked list each node points to the next node in the sequence; in a doubly linked list each node points both to the next and the previous node. This pointer-based organization allows for efficient insertion and deletion of elements at arbitrary positions (typically O(1) time when the node is known), without the need to shift other elements as in an array. However, accessing an element by index requires traversing the list from the head, giving O(n) time for random access.

5. RGC FRAMEWORK (Role–Goal–Context)

```
rgc_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a computer science instructor."),
    ("human", """
Role:
You are acting as a teacher.

Goal:
Explain binary search.

Context:
- Audience: Beginner
- Use simple examples
- Avoid heavy math
""")
])

chain = rgc_prompt | llm
print(chain.invoke({}).content)
```

Output

Binary Search – A Friendly Introduction

1. What Is Binary Search?

Imagine you have a **sorted** list of items (numbers, words, names …) and you want to know whether a particular item is in that list.
Instead of looking at every single element one-by-one (that's called *linear search*), binary search repeatedly cuts the list in half and discards the half that cannot contain the item.

Because we keep halving the search space, we find the answer **very quickly** – even for huge lists.

> **Key requirement:** The list must be **sorted** from low to high (or alphabetically). If it isn't sorted, binary search won't work.

---

2. The Core Idea – "Guess the Middle"

Think of the list as a book's index:

```
[ 2, 5, 8, 12, 16, 23, 38, 41, 56, 72 ]
```

You're looking for the number **23**.

1. **Look at the middle element**
   - The middle of 10 items is the 5th (or 6th) element.
   - Here the middle value is **16**.

2. **Compare** the middle value with the target (23)
   - 23 is **greater** than 16 → the target must be **to the right** of 16.

3. **Discard the left half** (everything ≤ 16) and repeat the process on the remaining right half:

```
[23, 38, 41, 56, 72]
```

4. Middle of this new sub-list is **41**.
   - 23 < 41 → discard the right half.

```
[23, 38]
```

5. Middle is **23** → **found!**

Each step cuts the remaining possibilities roughly in half.

---

3. Step-by-Step Walk-through (with a tiny list)

Let's use an even smaller list to see every decision clearly:

```
Sorted list: [3, 7, 9, 14, 20]
Target: 14
```

Step	Sub-list we're looking at	Middle index	Middle value	Comparison	New sub-list
1	[3, 7, 9, 14, 20]	2 (0-based)	9	14 > 9 → go right	[14, 20]
2	[14, 20]	0	14	14 = 14 → **found**	—

Only **two** checks were needed, even though the list has five items.

---

4. Why Is It Faster?

- **Linear search** checks each element one after another → worst-case = *n* checks (for a list of *n* items).
- **Binary search** halves the list each time → worst-case ≈ *$\log_2 n$* checks.

For a list of 1,000,000 numbers, binary search needs at most about **20** checks (because $2^{20} \approx 1{,}000{,}000$). That's a huge speed-up!

---

5. Pseudocode (Plain English)

```
function binarySearch(sortedArray, target):
    low  = 0                    // first index
    high = length(sortedArray)-1 // last index

    while low <= high:
        mid = (low + high) // 2   // integer division, middle index
        if sortedArray[mid] == target:
            return mid           // found! return its position
        else if sortedArray[mid] < target:
            low = mid + 1        // target is in the right half
        else:
```

```
            high = mid - 1        // target is in the left half

    return -1                     // not found
```

- `low` and `high` keep track of the current “window” we are searching inside.
- The loop stops when the window becomes empty (`low > high`).

---

6. Real-World Analogy

Finding a word in a dictionary

1. Open the dictionary roughly in the middle.
2. If the word you need comes **after** the page you’re on, you close the front half and open the middle of the remaining back half.
3. If it comes **before**, you do the opposite.
4. Keep repeating until you land on the exact page.

That’s exactly what binary search does, only with numbers (or any sortable items) in an array.

---

7. Quick Python Example (you can copy-paste and run)

```python
def binary_search(arr, target):
    low, high = 0, len(arr) - 1

    while low <= high:
        mid = (low + high) // 2          # middle index
        if arr[mid] == target:
            return mid                   # found, return position
        elif arr[mid] < target:
            low = mid + 1                # look right
        else:
            high = mid - 1               # look left
    return -1                            # not found

# ---- Demo ----
numbers = [2, 5, 8, 12, 16, 23, 38, 41, 56, 72]   # already sorted
print(binary_search(numbers, 23))   # → 5 (index of 23)
print(binary_search(numbers, 7))    # → -1 (not in the list)
```

```

**What you see:**
- The function returns the index where the target lives, or `-1` if it isn't present.

---

### 8. Things to Watch Out For

| Issue | Why it matters | How to avoid it |
|-------|----------------|-----------------|
| **Unsorted list** | Binary search assumes order; wrong results otherwise. | Always sort first (`list.sort()` in Python) or verify the list is already sorted. |
| **Integer overflow (in some languages)** | Computing `mid = (low + high) / 2` can overflow for huge indices. | Use `mid = low + (high - low) // 2` (the formula in the pseudocode). |
| **Off-by-one errors** | Mixing up inclusive vs. exclusive bounds (`low <= high` vs. `<`). | Stick to the pattern shown above and test with small arrays. |
| **Duplicates** | If the list contains the same value many times, binary search will find *some* occurrence, not necessarily the first or last. | For "first occurrence" you need a slight modification (keep searching left after a match). |

---

### 9. Recap – The Take-aways

1. **Binary search** works on **sorted** data.
2. It repeatedly looks at the **middle** element and discards half of the remaining items.
3. The number of checks grows **logarithmically**, so even massive lists are handled in a handful of steps.
4. The algorithm is easy to write in any language – just keep track of `low`, `high`, and `mid`.

Give it a try! Take a sorted list of your favorite numbers, pick a target, and manually walk through the steps on paper. You'll see how quickly the search space shrinks, and that's the magic of binary search. Happy coding!

# 6. Chain of thought

```
cot_prompt = ChatPromptTemplate.from_messages([
    ("system", """
You are a logical problem solver.
Think step-by-step internally.
Do NOT reveal raw chain-of-thought.
Provide a short structured explanation.
```
```

```
"""),
    ("human", """
If a function runs in O(n) time and is called n times,
what is the overall time complexity?
""")
])

chain = cot_prompt | llm
print(chain.invoke({}).content)
```

Output

Answer: \(O(n^2)\)

Reasoning

1. One execution of the function costs at most \(c \cdot n\) steps for some constant \(c\) (by the definition of \(O(n)\)).
2. The function is invoked \(n\) times, so the total work is at most
   \(n \times (c \cdot n) = c \cdot n^2\).
3. Ignoring constant factors, the overall running time grows proportionally to \(n^2\).

Hence the combined complexity is \(O(n^2)\).

7. Tabular format prompting

```
table_prompt = ChatPromptTemplate.from_messages([
    ("human", """
Compare the following data structures in a table:
- Array
- Linked List
- Stack

Columns:
- Structure
- Access Time
- Insertion Time
- Use Case
""")
])
```

```
chain = table_prompt | llm
print(chain.invoke({}).content)
```

Output

Comparison of Array, Linked List, and Stack

Structure	Access Time	Insertion Time*	Typical Use Case
Array (static or dynamic)	**O(1)** – direct indexing by position	**O(n)** in the worst case (shifting elements) – **O(1)** amortized for appending to a dynamic array (e.g., `ArrayList`, `vector`)	Storing a fixed-size collection where fast random access is required (e.g., lookup tables, matrix representations, buffers)
Linked List (singly or doubly)	**O(n)** – must traverse from the head to reach an element	**O(1)** if you already have a reference to the insertion point (e.g., at the head or after a known node); otherwise **O(n)** to locate the spot	Scenarios with frequent insertions/deletions in the middle of a sequence and where memory reallocation is undesirable (e.g., implementing queues, adjacency lists in graphs)
Stack (LIFO) – usually implemented with an array or linked list	**O(1)** for the top element (the only accessible element)	**O(1)** for `push` and `pop` operations (regardless of underlying implementation)	Managing recursive calls, undo/redo functionality, expression evaluation, backtracking algorithms, or any situation that needs last-in-first-out ordering

*Insertion time is expressed relative to where the insertion occurs:

- **Array:** Inserting at the end of a dynamic array is amortized **O(1)**; inserting elsewhere requires shifting elements → **O(n)**.
- **Linked List:** Inserting at the head or after a known node is **O(1)**; finding the insertion point first costs **O(n)**.

Key Takeaways

- **Arrays** excel at random access but can be costly for mid-sequence insertions.
- **Linked lists** provide cheap insert/delete at known positions but lack fast random access.
- **Stacks** are a specialized abstract data type focused on constant-time push/pop at one end, making them ideal for LIFO workloads.

8. Fill in the blank prompting

```
fill_blank_prompt = ChatPromptTemplate.from_messages([
    ("human", """
Fill in the blanks:
```

```
In Python, a _____ is used to store key-value pairs.
It is _____ (mutable / immutable).
""")
])

chain = fill_blank_prompt | llm
print(chain.invoke({}).content)
```

Output

```
In Python, a **dictionary** is used to store key-value pairs.
It is **mutable**.
```

9. Combined Practise Prompting

```
combined_prompt = ChatPromptTemplate.from_messages([
    ("system", """
You are a senior Coding AI.
Reason internally step-by-step.
Do not expose raw chain-of-thought.
"""),
    ("human", """
Role:
You are a DSA instructor.

Goal:
Explain time complexity classes.

Context:
- Audience: Beginners
- Output format: Table

Instructions:
- Keep explanations short
- Use examples
""")
])

chain = combined_prompt | llm
print(chain.invoke({}).content)
```

Output

Time-Complexity Cheat-Sheet

Complexity	What it means (in plain words)	Typical example (simple algorithm)
O(1)	Runs in constant time – the work does **not** grow with input size.	Accessing an array element `a[i]`.
O(log n)	Work grows logarithmically; each step cuts the problem size roughly in half.	Binary search in a sorted array.
O(n)	Work grows linearly with the number of items.	Scanning a list to find the maximum.
O(n log n)	Linear work *plus* a logarithmic factor; common for “divide-and-conquer” sorts.	Merge sort or heap sort.
O(n^2)	Quadratic growth – often from two nested loops over the same data.	Naïve bubble sort; checking all pairs in a list.
O(2^n)	Exponential growth – the work doubles for each extra element.	Generating all subsets of a set (power set).
O(n!)	Factorial growth – the work multiplies by the current size each step.	Brute-force traveling-salesperson (try every permutation).

Key tip: As **n** (input size) gets larger, the higher-order terms dominate. Aim for algorithms in the lower rows (O(1)–O(n log n)) for scalable solutions.